

Total Marks = 50

The objective is for you to write a program in C, getting some experience in the overall process. This includes designing the software modules according to the strict constraints given below, making flowcharts (before the actual coding) and performing complete testing (using only the lab machines). In assignment 2 you will be programming the same problem using the ARM assembly language, new to you, and using your C solution as a guide. Thus make sure you write a good program here as it will come back to haunt you or help you!

Part 1: The problem to be solved - Image processing

You are asked to do some image processing on an image composed of characters shaped in an $m \times n$ grid. For example, a 9×6 image may look like the one shown in Figure 1, which, more or less, looks like a representation of the letter “F” (you will not be using colour, however).

+	+	+	+	+	+
+	&	&	&	&	+
+	&	+	+	+	+
+	&	+	+	+	+
+	&	&	&	+	+
+	&	+	+	+	+
+	&	+	+	+	+
+	&	+	+	+	+
+	+	+	+	+	+

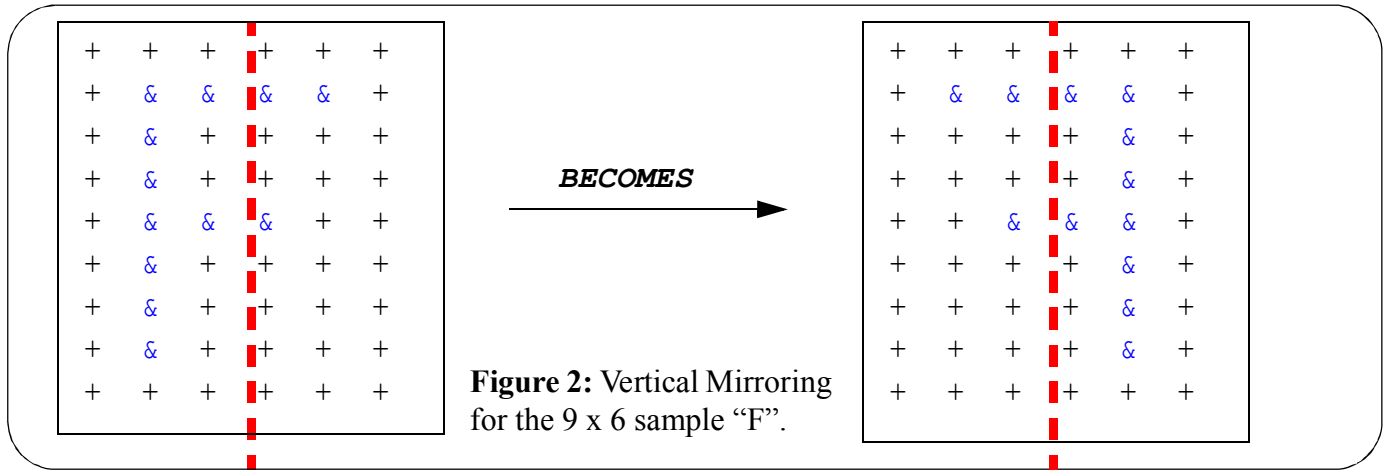
Figure 1: A 9×6 image containing the letter “F” described by “&” characters

Your program is expected to read from an input file an image described by integers, convert each integer appropriately to a character, do some image processing tasks on the character image, and print the results both to the screen and to an output file. This set of processing routines you are asked to implement are commonly found in a drawing program. The functional requirements are shown graphically and more formally below, together with further specifications for implementation. You will implement many subroutines, both functions and procedures, with detailed instructions on what parameters to use for some of them, while you will have more freedom of choice for others. Read the expectations carefully, both for the design and for the technical details.

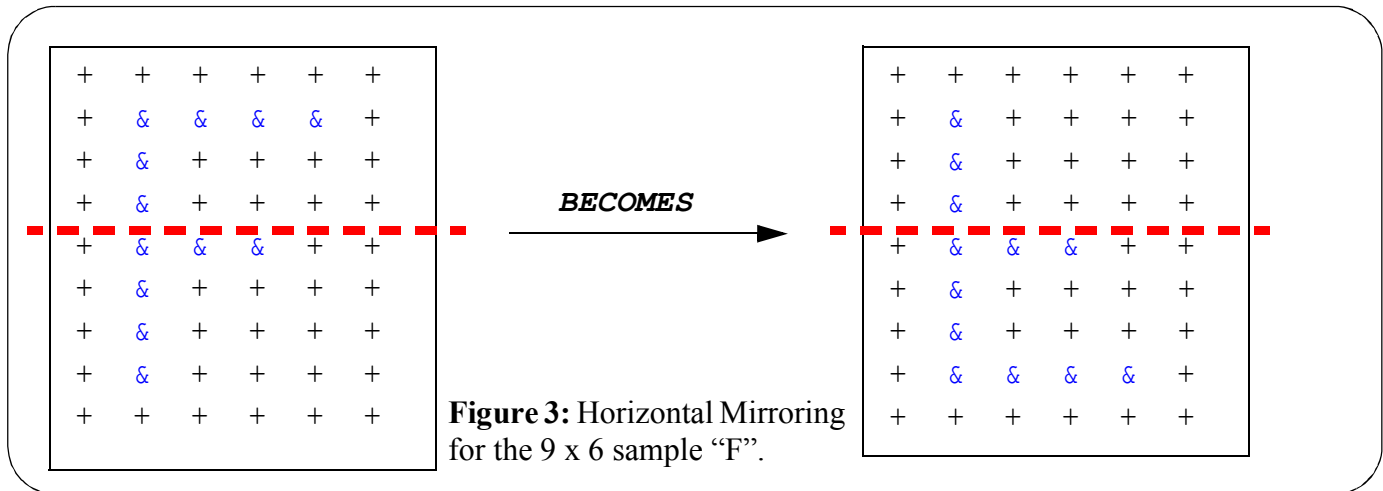
Part 2: The image processing functional requirements – illustrated

There are 5 possible tasks to be applied to a given image, upon request from the user: vertical mirroring, horizontal mirroring, diagonal right mirroring, diagonal left mirroring and rotation by 90 degrees.

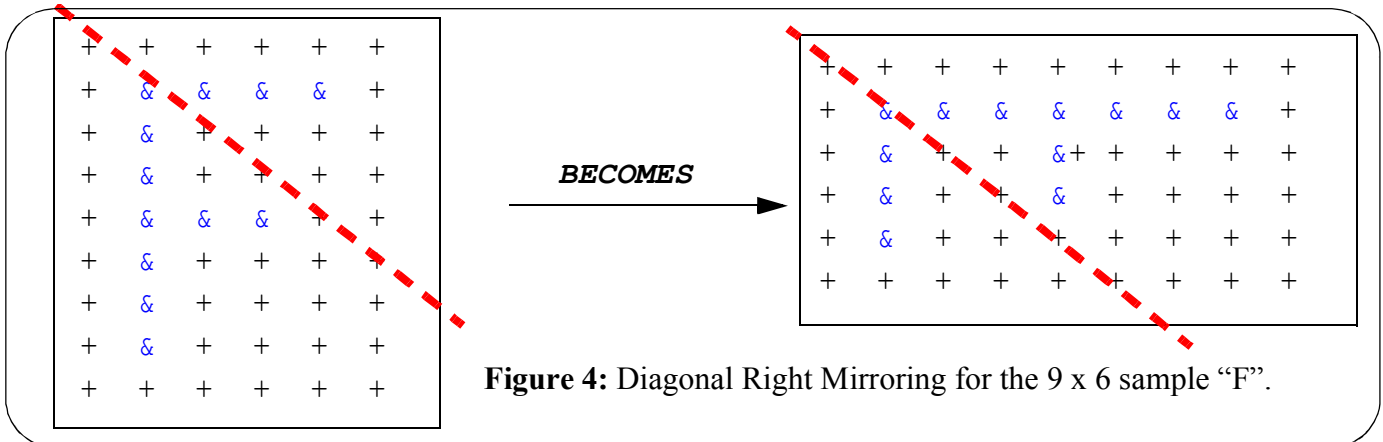
(a) Vertical mirror image: invert along a mid vertical axis. Figure 2 illustrates the task for the image “F”.



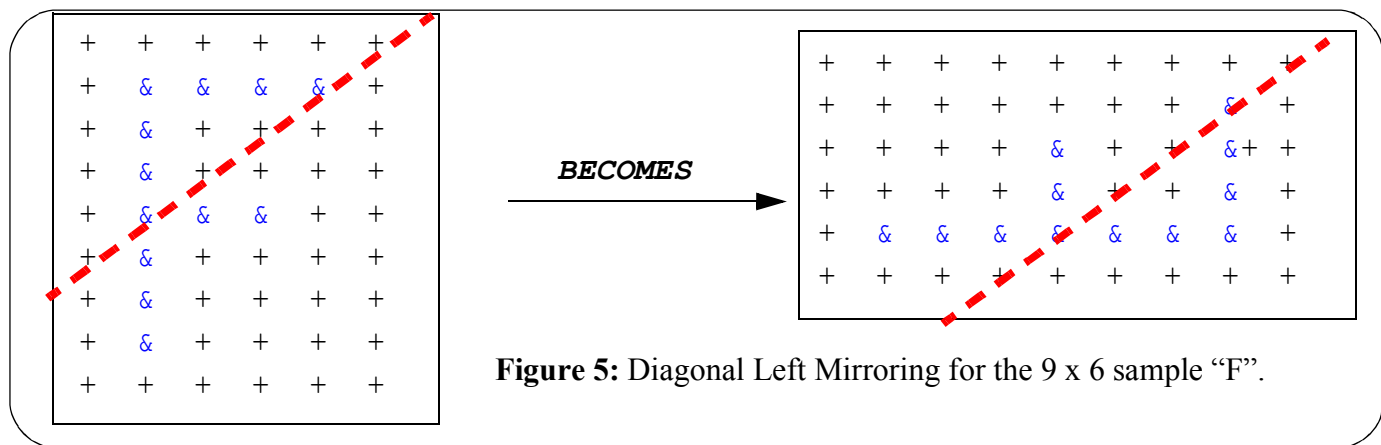
(b) Horizontal mirror image: invert along a mid horizontal axis



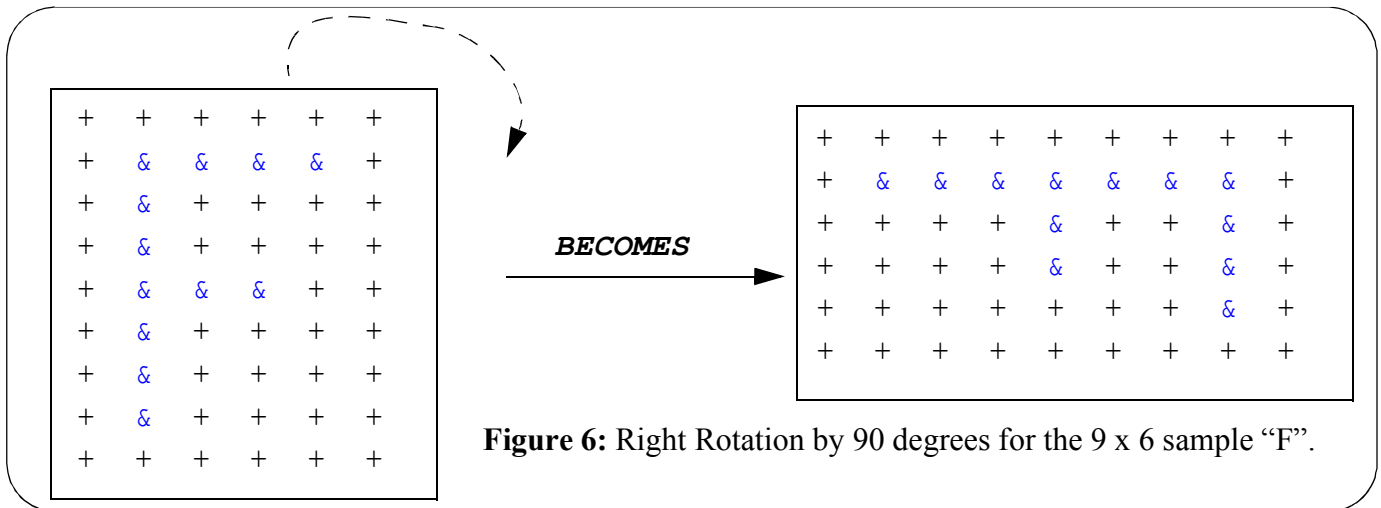
(c) Diagonal right mirror image: invert along the diagonal from top left corner towards the right.



(d) Diagonal left mirror image: invert along the diagonal from top right corner towards the left.



(e) Rotate right by 90 degrees.



Part 3: The overall requirements – pseudo-code and flowchart

The easiest way to summarize the flow of your application is to outline the overall functionality with pseudo-code, see Figure 7, or with a flowchart, see Figure 8. The detailed specifications are given below.

Figure 7: Pseudo Code

Initialization:

Open the input file
 if problems, print message and exit program
Open the output file
 if problems, print message and exit program
Print a header message to screen and to file

Obtain the input data:

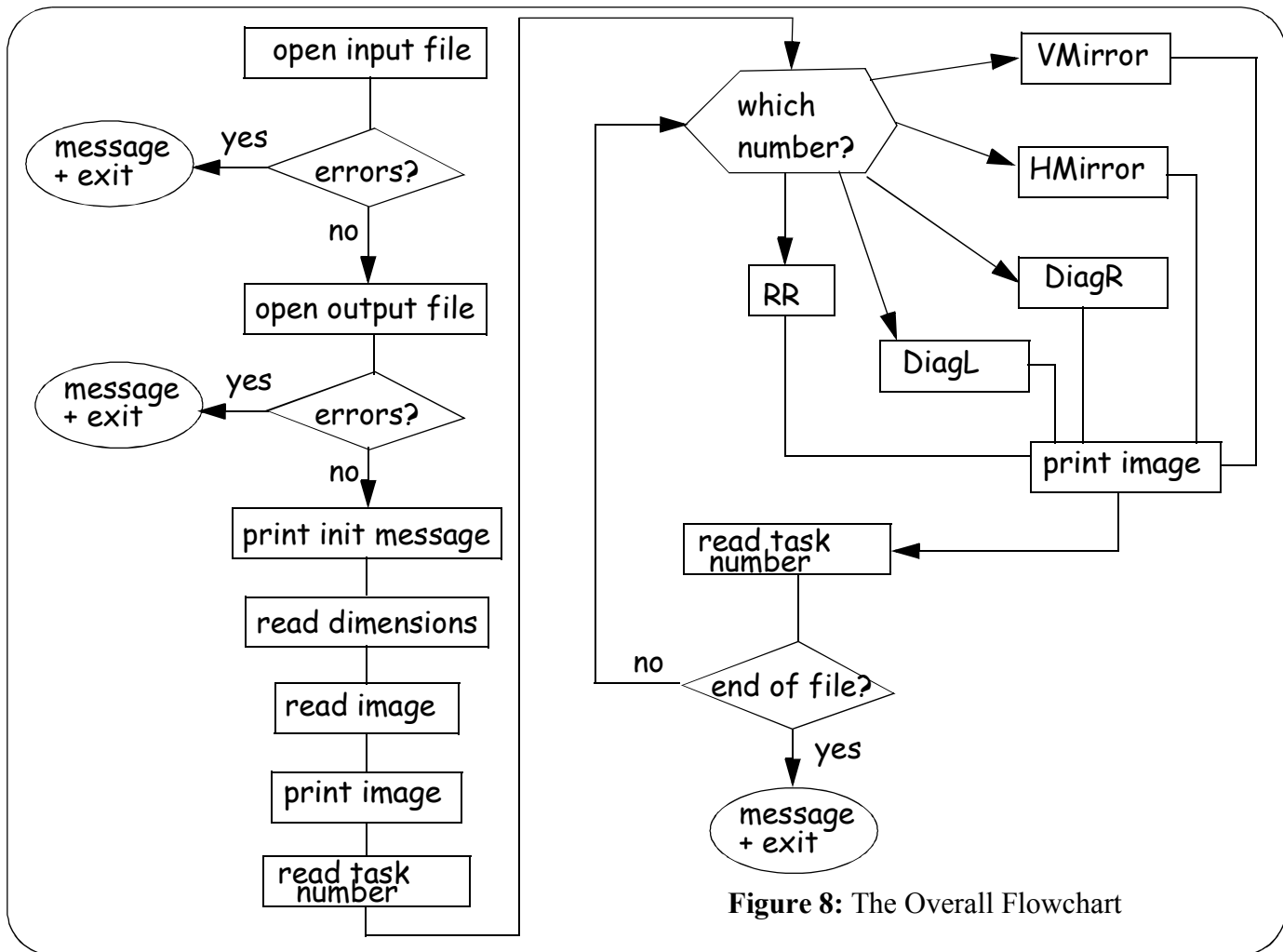
Read row size and column size of image
Read elements of image as integers, convert to characters, and store as 2D char array
Print image as characters with headings to screen and to file

Processing the image:

Repeat until the end of file is reached:
 Read an integer k from the input file specifying a task for the image
 Case k of:
 If 1 then call Vertical Mirror Image routine
 If 2 then call Horizontal Mirror Image routine
 If 3 then call Diagonal Right Image routine
 If 4 then call Diagonal Left Image routine
 If 5 then call Rotate Right Image routine (BONUS)
 Print the new image generated with headings to screen and to file

Closure:

When End of File is reached
 Print a final message to screen and to file
 Close input and output files
Exit the program



Part 4: The specifications

1. *Printing the image.*

The images are to be printed row by row as characters. The final requirement is for the printing to be done *both* to an output file and to the screen, but you may find it useful to print to the screen only at first for quick checking. Make sure that you read the documentation for printing “*characters*” to the screen and to an output file, printing “*strings*” to the screen and to an output file, versus printing “*integers*”, as the semantics are quite different. Use a procedure named: `void PrImage (Image, Nrows, Ncols)` where “Image” is the 2D character array, and “Nrows” and “Ncols” are the dimensions as integers. This procedure should be called in `main` when the initial image is printed and by `RdDoTask` when the final images, after processing, need to be printed. The call would be, for example, “`PrImage (IMchr1, Rsize1, Csize1)`” when the initial image is printed.

2. *Opening input and output files.*

Instructions on how to open and close input and output files are available in the document posted in the Software Resources web pages of the course under the title: “*I/O in C*”. All this can be done within the main routine, without any extra functions (at least for this initial assignment). You only need to check for errors in opening the files, but not for errors in finding an early end of file. That is, you are guaranteed that once you start reading successfully, at least two integers for the dimension of a 2D array, one whole 2D array and at least one image processing command will be available correctly. Normally a larger set of inputs will be present. It is a good idea to experiment with a set of input files with images of various sizes, without any image processing operations. One such files, called “ImageF1.txt” will be posted on the

assignments page, but do not assume that this will be the file used in testing your code for evaluation. No parameters must be passed through the command line for file names or any options. The input and output file names will be fixed in your program (as given below). Also examine carefully the initial template file given to you, called “AlTemplate.c” as some of the code for this part is supplied.

3. *Reading in the sizes.*

The sizes are given as two positive integers, no bigger than 50. Store them in two variables called “Rsize1” and “Csize1” for the initial image. You are not expected to do any input validation. This implies that you do not need to check that these numbers are positive and less than 50 – you are guaranteed that they will be in range. This is a valid assumption because normally one would put a set of input data through a preprocessing stage to clean up any invalid data as a separate step. Use a procedure named `void RdSize(*Nrows, *Ncols)` to read the dimensions, invoked in the main routine. The call to `RdSize` passes in `Rsize1` and `Csize1` as the two input parameters by reference, since their content will be changed. There is no output parameter. Use the names `Nrows` and `Ncols` for the corresponding binding parameters in the procedure itself, following the same style as for the other routines (see more examples in “AlTemplate.c”).

4. *Reading in the image.*

The image is given in the input file as a set of integers which can only be “1” or “0”. You are guaranteed that the correct number is present, that is, there are exactly $Rsize1 \times Csize1$ integers in the file. They do not need to be stored anywhere (see below for transformation into characters). Use a procedure:

```
void RdImage(Image, Nrows, Ncols),
```

where `Image` is the 2D array of characters in memory allocated for your image and “`Nrows`” and “`Ncols`” are the dimensions as integers. This procedure should be called by the main routine as:

```
RdImage(IMchr1, Rsize1, Csize1).
```

A requirement on naming for consistency: name the original image array “`IMchr1`” and the array for the second image produced by any task of your image processing operations “`IMchr2`”. Name the dimensions “`Rsize1`”, “`Csize1`” respectively.

5. *Transforming the image to characters.*

Transform each integer read from the file into a 1-byte character, as: a “1” becomes a “&” and a “0” becomes a “+”. Store the characters into a two-dimensional array of characters, indexed for use by `Rsize1` rows and `Csize1` columns. All this transformation work should be done from within the procedure `RdImage` introduced above. Thus there is no need to allocate a 2D array of integers. You should allocate a 2D array of characters of size 50 x 50, which is the maximum you would ever deal with. No dynamic allocation is expected *or allowed*. You will use the portion of the 50 x 50 static arrays by indexing, as necessary given the actual given sizes of the input image.

6. *Reading the commands for processing.*

You should implement reading a set of integers in a loop and test for end of file. You also have to implement the corresponding “switch” control structure applied to each value after reading it in. Use a function as:

```
int RdDoTask(Image1, Image2, Nrows, Ncols)
```

which returns a “0” if no end of file was reached and a “1” if end of file was detected. It is called by the main routine as:

```
EOF = RdDoTask(IMchr1, IMchr2, Rsize1, Csize1)
```

`RdDoTask` in turns calls the appropriate subroutines corresponding to the required task. There are up to 5 possible processing operations, which are numbered 1 to 5 as follows and whose expected function name is listed on the right:

1	Vertical Mirror	VMirror
2	Horizontal Mirror	HMirror
3	Diagonal Right	DiagR
4	Diagonal Left	DiagL
5	Rotate Right 90 degrees	RR

These numbers can appear in *any order* in the input file and at least one of them is guaranteed to be in the file. Each of these processing operations are applied to the original image, “IMchr1”, as read in to construct a new image, “IMchr2”, which will be printed. RdDoTask also calls PrImage after each image processing task has been completed.

7. Names for the input and output files

The input file name must be hard-coded as: “A1In.txt”. The output file name must be hard-coded as: “A1Out.txt”. Substitute the content with whatever tests you want to supply, including the one posted.

Part 5: Implementing the image processing operations through appropriate routines

For each image processing routine, here is a more detailed explanation of its requirements and specifications. Note the interface shown which should give you an idea of the expectations for good documentation in any programming language and absolutely so in this assignment. The parameters are listed, a summary of what a function is supposed to do, and, when necessary, a summary of how it is supposed to do it (the algorithm).

1. Vertical Mirror

```

/***** void VMirror(Image1,Image2,Nrows,Ncols) *****/
/*Given the 2D char array of Image1 and its dimensions,
construct the vertical mirror image in Image 2 as in:
    copy columns (0,1,...,Ncols-1) of Image1 to
    columns (Ncols-1, Ncols-2, ..., 1, 0) respectively of Image2 */
void VMirror(char Image1[MAXROW][MAXCOL],char Image2[MAXROW][MAXCOL],
    int Nrows,int Ncols)
{ your code here
} /*End of VMirror*/

```

2. Horizontal Mirror

```

/***** void HMirror(Image1,Image2,Nrows,Ncols) *****/
/*Given the 2D char array of Image1 and its dimensions,
construct the horizontal mirror image in Image 2 as in:
    copy rows (0,1,...,Nrows-1) of Image1
    to rows (Nrows-1,Nrows-2,...,1,0) respectively of Image2 */
void HMirror(char Image1[MAXROW][MAXCOL],char Image2[MAXROW][MAXCOL],
    int Nrows,int Ncols)
{ your code here
} /*End of HMirror*/

```

3. Diagonal Right

```

/***** void DiagR(Image1,Image2,Nrows,Ncols) *****/
/*Given the 2D char array of Image1 and its dimensions,
construct the flipped image in Image2 along the top
left to bottom right diagonal as in:
    copy col 0 of Image1 -> row 0 of Image2
    copy col 1 of Image1 -> row 1 of Image2
    .....
    copy col (Ncols-1) of Image1 to row (Ncols-1) of Image2
    NOTE: sizes of Image2 are inverted from Image1 */
void DiagR(char Image1[MAXROW][MAXCOL],char Image2[MAXROW][MAXCOL],
    int Nrows,int Ncols)
{ your code here
} /*End of DiagR*/

```

4. Diagonal Left

```

/***** void DiagL(Image1,Image2,Nrows,Ncols) *****/
/*Given the 2D char array of Image1 and its dimensions,
construct the flipped image in Image2 along the top
right to bottom left diagonal as in:

```

```

        copy col (Ncols-1) of Image1 -> row 0 of Image2
        copy col (Ncols-2) of Image1 -> row 1 of Image2
        .....
        copy col 0 of Image1 -> row (Ncols-1) of Image2
        NOTE: sizes of Image2 are inverted from Image1 */
void DiagL(char Image1[MAXROW][MAXCOL],char Image2[MAXROW][MAXCOL],
        int Nrows,int Ncols)
{ your code here
} /*End of DiagL*/

```

5. Rotate Right by 90 degrees

You do not need to implement the functionality for this image processing, but you need to take care of the possible case in the “switch” control structure when reading the command “5” from a file. It is sufficient for this case to print a message as in: “*Rotation right by 90 degrees is not yet implemented*”. This is a bonus part of the assignment and a full correct implementation, including designing your own algorithm, will earn an extra 5 marks, but *only* if the other four routines are working correctly!

Part 6: Notes for good modular design

Extra functions. For a better implementation, it is useful sometimes to write extra routines to do common little jobs, separately. You will learn more about modularity and similar design issues in your software engineering classes as well. Here it is *strongly* suggested to have the following routines as well (but they are not required).

- (A) /* void CopyRow(Image1,Image2,Ncols,Rowi,Rowj) ==> used by HMirror
 Copy row i of Image1 to row j of Image2 */
- (B) /* void CopyCol(Image1,Image2,Nrows,Coli,Colj) ==> used by VMirror
 Copy col i of Image1 to col j of Image2 */
- (C) /*void CopyColRow(Image1,Image2,Nrows,Ncols,Coli,Rowj) ==> used by DiagR
 Copy col i of Image1 to row j of Image2 */
- (D) /*void CopyColrevRow(Image1,Image2,Nrows,Ncols,Coli,Rowj) ==> used by DiagL
 Copy col i from (Nrows to 0) of Mat1 to row j of Mat2 from 0 to Ncols */

This kind of structure can make design, coding and, most of all, testing and debugging, much easier.

Global variables. In summary, NO global variables should be used. The exception are the necessary file handles for opening and closing files, and for reading and writing to from/to files. Otherwise all other information needed by a function or procedure should be passed through its parameters. You will be penalized for using global variables.

Printing characters and strings. A *character* is a 1-byte entity. A *string* is defined as a sequence of characters. A string of n characters is implemented in C as an array of $n+1$ bytes with the rightmost byte containing the “0x00” NULL byte to denote the end of the sequence. Make sure you use the correct semantics in your instructions.

Input and Output Examples. Look at the web pages for extra files posted for examples of input files to aid you in testing your program, but do not hesitate to build up your own. There will also be examples of output as produced by an implementation to give you an idea of the expectations. Finally, do not forget about good documentation throughout, that is, meaningful and logical comments, not simply stating the sometimes obvious semantics of each line of code.

Part 7: A systematic approach

Here are some strong guidelines to acquire a good and a systematic approach using “*stub routines*”. The work below represents how your instructor would work at this task.

Phase 1: output and image processing routines using a hard coded small example (do this before Lab 3)

- (a) Start with the output functions. Do the input functions last. You always need to echo out parts of your computation as you go, so having a well tested output routine is essential. For the input, instead, it is much easier, while developing your program and testing, to initialize internally a sample array. Look at the really small one in the `Altemplate.c` file posted. Create a “main”, allocate and initialize (by hard coding) a sample 2D array of characters, and program and test only the procedure: `void PrImage (Image, Nrows, Ncols)`.
- (b) Design and program one of the four functions required for image processing, your choice. For example, start by designing and implementing: “`void VMirror(Image1, Image2, Nrows, Ncols)`” and test it on the pre-initialized 2D character array as above. Print out the result and learn well from the experience. Document your code, check all the parameters and then go and have a rest break.
- (c) Continue by designing one by one the other 3 image processing functions and test them on the same initialized 2D array as above. Test carefully by recoding a couple of other 2D arrays (e.g. a square one and another rectangular one for example).
- (d) Every time you increment your implementation, say by adding a new image processing task, save the previous source file with a version name and start working on a copy with the new version name. During implementation, normally one would start with `Alcsc230v1.c`, then `Alcsc230v2.c`, `Alcsc230v3.c`, and so on. If at some point things start failing, you can always go back to a well tested state in the previous source code. Keep all your version files in case you need to come for help.

Phase 2: reading integers from a file until end of file

- (a) Completely independently now, construct a simple input file called for example “`Switch.txt`” containing only the four numbers 1, 2, 3, 4 and 5 in some order. Now implement a separate small program containing only the control structure (loop) which opens this file, reads from it one number at a time until end of file and prints them out. That’s all. This way you now learned to open a file for input, read from it, detect end of file and do some easy printing.
- (b) Now increment the reading of the 5 numbers (from a file in their loop structure until end of file) with an appropriate call to a stub image processing function corresponding to the value, using a “switch” statement. In this first pass, the “switch” control structure simply prints a message with the task number - remember, you are testing the control logic, not the image processing task. When you are satisfied that it all works, then integrate this second test program with your previous one from Phase 1 containing the single pre-initialized 2D array. At this point you should be testing a working program which initializes a 2D array of characters, opens a file, reads a series of commands (integers), executes appropriately the image processing task correlated to the commands and prints out to screen and to a file the processed new image.

Phase 3: reading the data for the image from a file

- (a) You have left it until now to do any implementation of reading an input file containing the data for the 2D array. You will get some practice in the Lab session for this, so it may be worth waiting for. You know how to open a file for input and read numbers (you have done it above in Phase 2). Now you must design and implement the two procedures, “`void RdSize(Nrows, Ncols)`” to get the sizes of the image, and `void RdImage (Image, Nrows, Ncols)` which transforms each integer read in (0 or 1) into the appropriate character and stores it into the 2D character array. It might be useful to write a completely separate program just to do this part: read the sizes, read/convert/store the image and print it out, without doing any task or reading further commands from an input file.
- (b) Finally put it all together and test it.

The systematic approach above looks tedious and you all probably think you can do better than that (and faster) using your own brilliant methodology. Trust the instructions above, no matter how pedantic they look - they work as they divide the work into manageable and easily testable modules, and they give an ordering, possibly non-intuitive, to the tasks you need to do.

Part 8: Documentation

1. Your source code file *must* start with the following items, shown by example:

```
/*    V00123456 Micaela Serra
      Alcsc230.c      September 2011
      CSC 230, Assignment 1, Part 2 */
/* This program implements...a short description here ... */
```

2. Every procedure or function *must* start with the following items, shown by example:

```
/**** int RdDoTask(Image1,Image2,Nrows,Ncols) ****/
/*This function is called by main to read one integer from an
open input file. Based on the value read, it calls the
appropriate image processing routine and then prints the
new image. It returns to main with a "0", unless end of file
was encountered, in which case it returns a "1", without
doing any processing */
/*INPUT PARAMETERS:
    Image1 and Image2: 2D arrays of characters
    (initial and processed images)
RETURN PARAMETER:
    "1" when end of file encountered, "0" otherwise*/
int RdDoTask (char Image1[MAXROW][MAXCOL],
              char Image2[MAXROW][MAXCOL],int Nrows, int Ncols).....
```

That is, every subroutine must have a clearly readable start point, followed by a clear explanation of its purpose, followed by documentation of its parameters and finally its initial header, before any executable instructions appear.

3. All through the coding meaningful comments are expected, especially in points where it is not crystal clear what is happening, unless one deciphers carefully the instructions. It is sufficient to state something like: "*Copying row i of Array1 to row j of Array 2*", as long as what is happening is evident.

Part 9: Evaluation

Correct execution of your program will gain you at most 50% of the total marks. This implies as well that there are no compiler warnings! The other 50% of the evaluation is based on the quality of your code which will be analyzed for design, structure, documentation, clarity, organization, modularity.

Part 10: What to hand in

1. Your working implementation in a file named "Alcsc230.c" by electronic submission through the Connex site.
2. A printed copy of your source code as listed in "Alcsc230.c" in the CSC 230 box in the ECS building, 2nd floor. If no printed copy is available, only 50% of the marks (for execution) will be given.

Part 11: The Initial Template : AlTemplateFall.c

To help you out on the correct path, an initial file called "AlTemplateFall.c" is posted for you. Make sure you understand the code in this file really well, and then proceed to make new versions of it, in separate

files, with your new code. Check the web pages often, attend the labs and the lectures, as more examples and information to help you along will become available.

It is strongly suggested that you complete phases 1 and 2 before Quiz #2 in Lab 3. That lab will help you for phase 3.

Part 12: Sample output

For the sample input file posted on the Connex site, namely “A1inF.txt” which corresponds to the illustration in Part 2, here is the output of my program. Note the opening and closing statements and the appropriate labeling of each step.

Figure 9: Sample output for input file “A1inF.txt”

Hello:

Captain Picard - Student Number V00123456

File = A1csc230.c- Fall 2011

CSC 230, Assignment 1, Part 2

Starting:

Initial IMchr1 contains:

```
+ + + + + +
+ & & & & +
+ & + + + +
+ & + + + +
+ & & & + +
+ & + + + +
+ & + + + +
+ & + + + +
+ + + + + +
```

TASK 3 = Diagonal Right

IMchr2 contains:

```
+ + + + + + + + +
+ & & & & & & +
+ & + + & + + + +
+ & + + & + + + +
+ & + + + + + + +
+ + + + + + + + +
```

(continued next page)

(Continued): Sample output for input file "A1inF.txt"

TASK 1 = Vertical Mirroring

IMchr2 contains:

```
+ + + + + +
+ & & & & +
+ + + + & +
+ + + + & +
+ + & & & +
+ + + + & +
+ + + + & +
+ + + + & +
+ + + + + +
```

TASK 4 = Diagonal Left

IMchr2 contains:

```
+ + + + + + + + +
+ + + + + + + & +
+ + + + & + + & +
+ + + + & + + & +
+ & & & & & & +
+ + + + + + + + +
```

TASK 2 = Horizontal Mirroring

IMchr2 contains:

```
+ + + + + +
+ & + + + +
+ & + + + +
+ & + + + +
+ & & & + +
+ & + + + +
+ & + + + +
+ & & & & +
+ + + + + +
```

TASK 5 = Rotation Right

Optional task 5 - not implemented

The program is all done - Bye!